

Reinforcement Learning

Class 1

***Multi-armed bandits:
Exploration, Exploitation, and Learning from Interaction***

Most material adapted from Sutton & Barto's book, chapter 2

Introduction

- Most important feature distinguishing Reinforcement Learning: uses training information that *evaluates* the actions taken rather than *instructs* by giving correct actions
- Evaluative feedback depends entirely on the action taken, whereas instructive feedback is independent of the action taken
- In this class, we study a simplified *nonassociative setting*
- We explore a simple version of the *k-armed bandit problem*
- We introduce a few basic learning methods, which we extend in later classes to apply to the full reinforcement learning problem

Motivation

- Allow to explore some ideas, mainly on **exploration vs exploitation**, on a simple setting (single state)
- Multi-armed bandits model **sequential decision-making under uncertainty** when we must **learn while acting**
- Real world **applications**:
 - Ads – choosing which add to display
 - Recommender systems (contextual multiarmed bandits)
 - Adaptive A/B testing
 - Clinical trials (adaptive)
 - ...

K-armed bandit problem

- The agent needs to choose among **k different actions**
- After each choice, the agent receives a numerical reward chosen from a **stationary probability distribution** that depends on the selected action
- The objective is to **maximize the expected total reward** over several time steps (at each time step the agent makes a decision)
- Each of the k actions has an expected (mean) reward given that the action is selected - **value** of that action
- Value of an action a in time t – expected reward (R_t) given action a is selected (unknown)

$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a]$$

K-armed bandit problem

- If we **knew** the value of each action, it would be trivial to solve the problem: **always select the action with highest value**
- We assume that the agent **does not know the action values** with certainty, although it may have **estimates**.
- So, we will **estimate** the value of action a - $Q_t(a)$ - and try to approximate the ideal value

Exploration vs exploitation

- If we select at time t the action with the highest estimated value Q_t , this is a **greedy** action and we are **exploiting** (using the best possible judgment to have the highest expected reward)
- If we select another action we are **exploring** – allows to improve the estimate of other actions (and hopefully to find better ones)
- Exploitation “assures” the best possible outcome in the short term; exploration may allow better rewards in the long run
- Since at a given time step we can only do one there is a “conflict”, and we need to devise a **strategy** to select when to exploit and when to explore – this is a core RL dilemma !!

Action-value methods

- **Action-value methods** allow to:
 - **estimate** the values of actions (mean reward)
 - use the estimates to make **action selection decisions**
- Simplest way to estimate – averaging the rewards received until t – **sample average method**

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}}$$

- Simplest action selection rule – **greedy** - select the action with the highest value of $Q_t(a)$ (or one of the actions if there are ties)

$$A_t \doteq \arg \max_a Q_t(a)$$

Action-value methods

- Greedy method - always **exploit** – might be suboptimal stuck in an action and not improving estimates of others
- Alternative algorithm – **ϵ -greedy method** – with a small probability ϵ , select another action (randomly)

Full algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Loop forever:

$$A \leftarrow \begin{cases} \operatorname{argmax}_a Q(a) & \text{with probability } 1 - \epsilon \\ \text{a random action} & \text{with probability } \epsilon \end{cases} \quad (\text{breaking ties randomly})$$

$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

Update rule

Saves memory
and computation

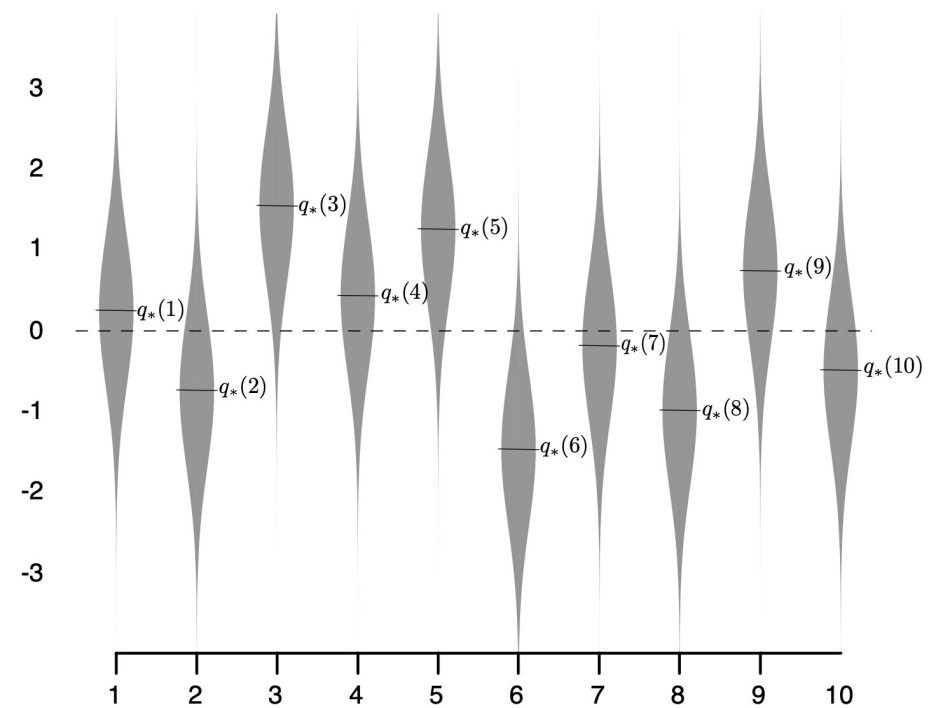
$$\begin{aligned}Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\&= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\&= \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \\&= \frac{1}{n} \left(R_n + (n-1) Q_n \right) \\&= \frac{1}{n} \left(R_n + n Q_n - Q_n \right) \\&= Q_n + \frac{1}{n} \left[R_n - Q_n \right],\end{aligned}$$

General form

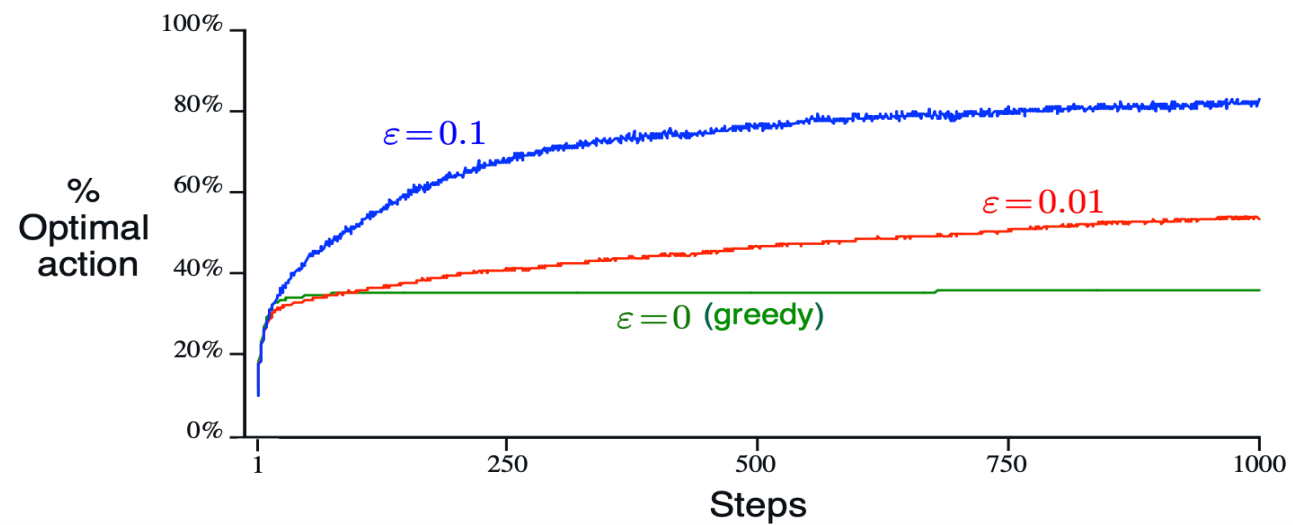
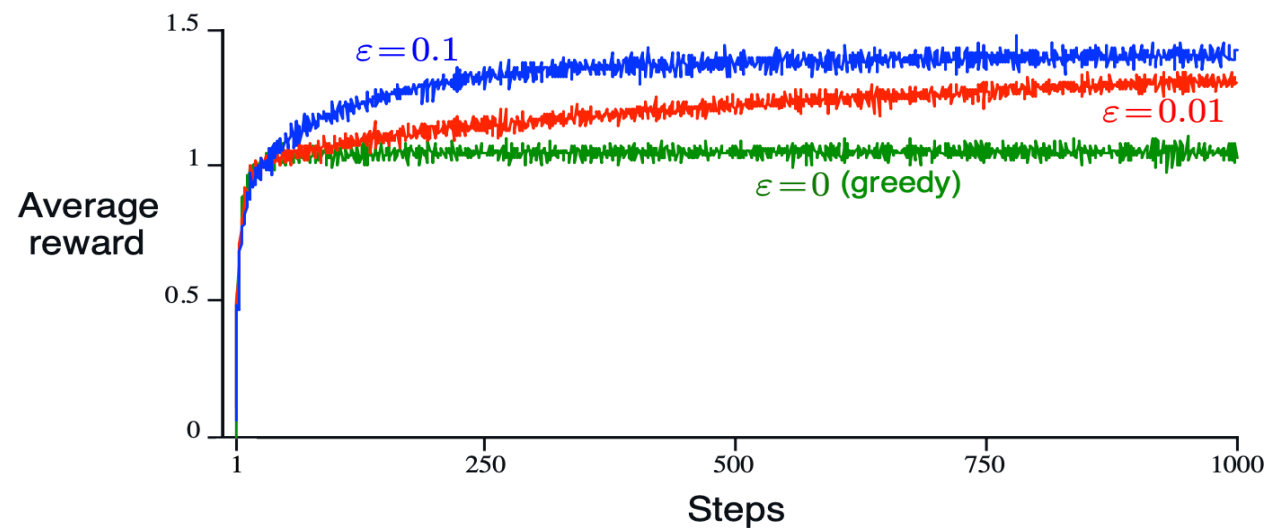
$$NewEstimate \leftarrow OldEstimate + StepSize \left[Target - OldEstimate \right]$$

Benchmark testbed

- Simple benchmark scenario – 10 armed testbed
- Generate a large number **N** of k-armed bandit problems with $k = 10$ (typically $N \geq 1000$)
- For each problem: action values $q_*(a)$ are selected from $N(0,1)$ distribution
- When an action a is selected – the reward is taken from a distribution with mean $q_*(a)$ and variance 1
- Each experiment runs for T steps and a run is done for each of the N scenarios



Example results



From Sutton & Barto book

Nonstationary environments

- What if action values change along time ? **Non-stationary environments** – more realistic
- It makes sense to give more weight to recent rewards than to long-past rewards

$$Q_{n+1} \doteq Q_n + \alpha [R_n - Q_n] \quad \alpha \in (0, 1] \text{ is constant.}$$

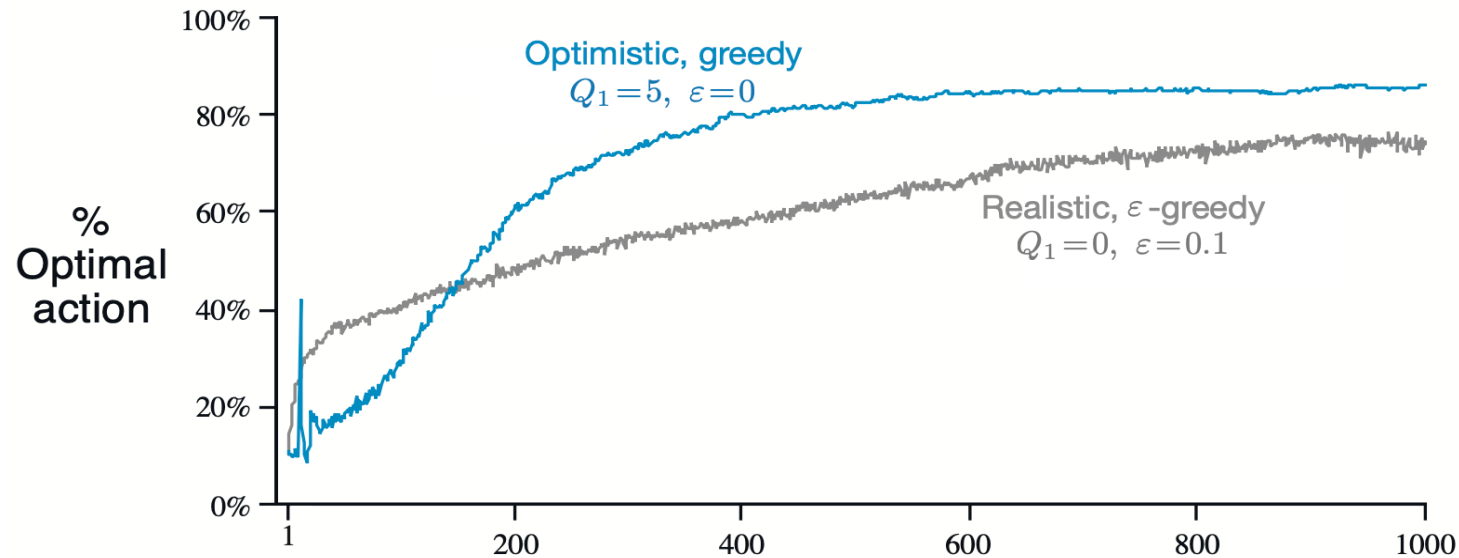
- For a time i in the “past”, the weight given to R_i decreases as the number of intervening rewards $(n-i)$ increases

Optimistic initial values

- Methods described before rely heavily on the first estimate – $Q_1(a)$
- For the sample average method, we can eliminate bias once all actions have been selected once
- For methods with constant α , the effect of the initial value reduces over time but is not immediately eliminated
- Initial values become parameters that need to be set by users
- One option is to set initial estimates to be overly **optimistic**, i.e. higher than a very large proportion of possible values
- Optimistic values will “force” all actions to be tested in the early stages – **encourage exploration in the beginning**

Optimistic initial values

- May show good results in stationay environments



- Not well suited for non-stationary environments

Implementation

- Let's implement the algorithms mentioned until now !!
- There is a core Python file to start the implementation
- Class ***KArmedBandit*** defines the benchmark specified above
- Class ***BanditAgent*** represents the base class for the agents
- Class ***EpsilonGreedy*** extends the agent to implement the previous method ... **please complete it !**
- Function ***run_experiment*** will be used to run experiments !
- A set of functions allow to reproduce some results shown here !!

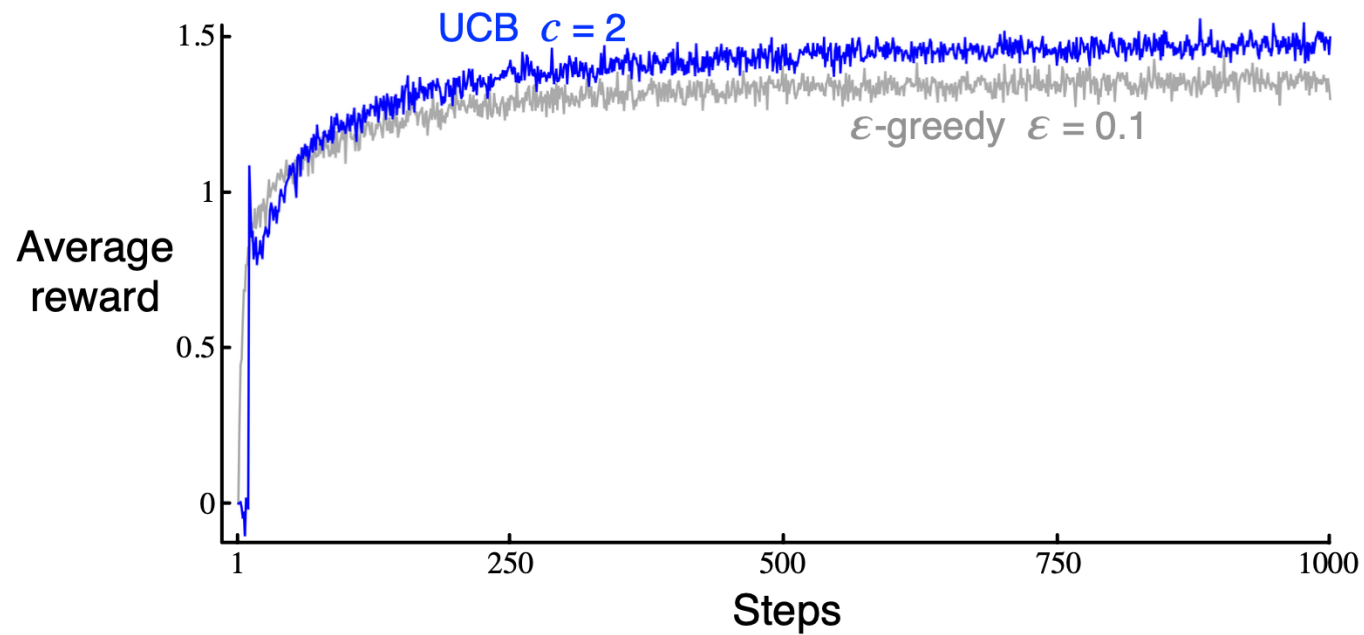
Upper confidence bounds

- Exploration is needed because there is always **uncertainty** about the accuracy of the action-value estimates
- But, maybe not all (nongreedy) actions should be treated the same
- One strategy could be to select among the non-greedy actions according to their potential for actually being optimal by adding a **measure of “uncertainty”**

$$A_t \doteq \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

- The square-root term is a measure of the uncertainty or variance in the estimate of a's value; $c > 0$ controls the degree of exploration

Upper confidence bounds



Illustrative results from Sutton & Barto's benchmark

Implementation part II

- Class **UCB** extends the agent to implement the previous method
... **please complete it** !

Gradient algorithms

- One alternative to the previous is learning numerical preferences for each action a - $H_t(a)$

$$\Pr\{A_t = a\} \doteq \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}} \doteq \pi_t(a) \quad \text{Softmax function}$$

Probability of taking action a at time t

- Idea similar to Roulette Wheel on Evolutionary Algorithms

Gradient algorithms

- Update of H based on the idea of **stochastic gradient ascent**:

$$\begin{aligned} H_{t+1}(A_t) &\doteq H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)), & \text{and} \\ H_{t+1}(a) &\doteq H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a), & \text{for all } a \neq A_t \end{aligned}$$

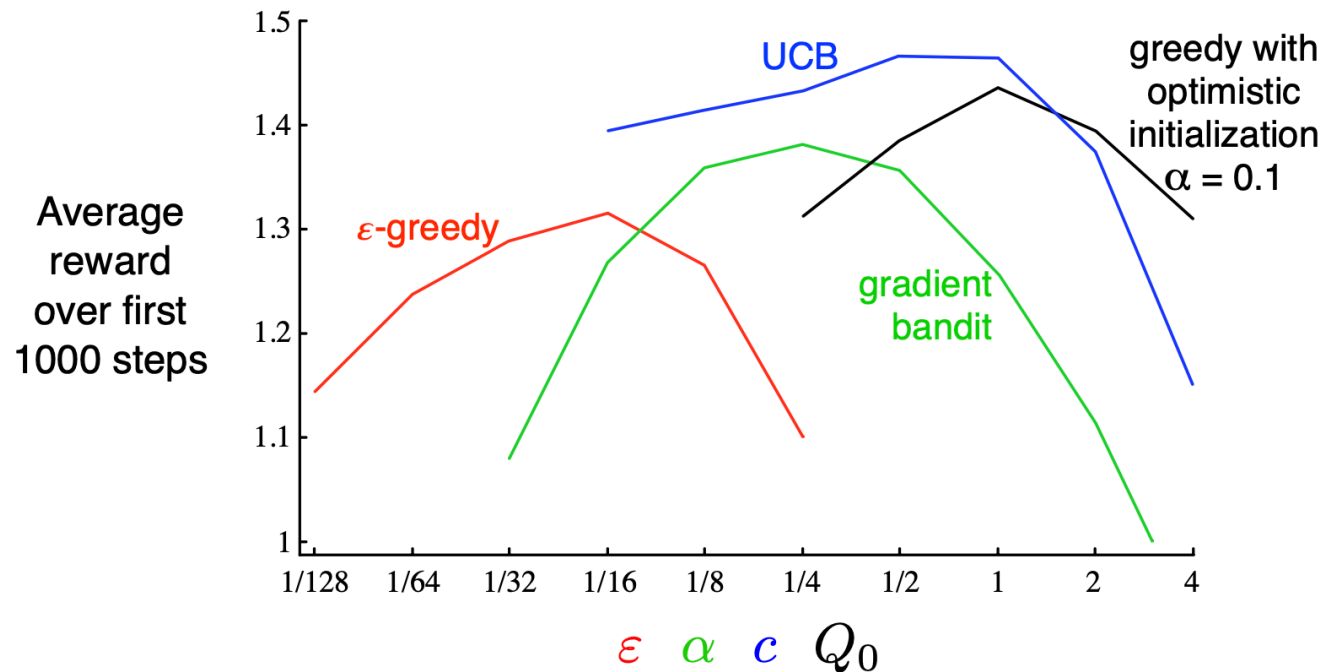
$\alpha > 0$ is a step-size parameter, and $\bar{R}_t \in \mathbb{R}$ is the average of all the rewards

$\bar{R}_t$ term serves as a baseline with which the reward is compared

- If the reward is higher than the baseline, then the probability of taking A_t in the future is increased, and if the reward is below baseline, then probability is decreased. The non-selected actions move in the opposite direction

Summary: which method is best ?

- The answer is very dependent on context and parametrization ...



Above a parametric study from Sutton & Barto's book

Implementation part III

- Class **GradientBandit** extends the agent to implement the previous method ... **please complete it** !

Contextual bandits

- The scenarios above do not consider different situations/ contexts, i.e. all decisions are made in a single context
- In a general reinforcement learning task there is more than one situation, and the goal is to learn a **policy**: mapping from situations to the actions that are best in those situations/ contexts
- A different scenario would be: different k-armed bandit tasks, and on each step you confront one of these
- If each context is signalled by a “clue” - **associative search task (contextual bandits)** - intermediate between the k-armed bandit problem and the full reinforcement learning problem (If actions affect the next situation + the reward, we have the full reinforcement learning problem)